

Activity Is Not Progress: Which Runtime Signals Reveal That a Coding Agent Has Stalled?

Qichen Tong Ke Chuang Jiang

September 11, 2026

Abstract

Coding agents spend much of their runtime on activity that does not move the task forward. We ask which signals an online, reference-free monitor can use to notice this, and we answer it with annotated evidence rather than a new model. Progress is defined through three channels — epistemic (new task-relevant information), implementation (durable plausible changes) and verification (objective evidence about correctness) — and we label 654 windows over 81 trajectories with a written codebook, then re-label them with a different reader under reduced context to measure stability (88.1% binary agreement, $\kappa = 0.70$). Using 1500 Terminal-Bench 2.0 trajectories (45 tasks, 9 scaffolds, 201696 online feature vectors) evaluated with task-level cross-validation, we compare six signal families: action repetition, relevance-blind novelty, task-grounded evidence, verification deltas, workspace churn, and rolling semantic redundancy of successive steps.

The result is not the one we expected. Rolling semantic redundancy is the strongest single family (global ROC-AUC 0.742, within-trajectory 0.678), and task-grounded evidence on its own does *not* beat it (0.620; paired Δ over trajectories -0.129 with a clustered 95% interval that excludes zero). Evidence reaches parity with the semantic baseline only when the two are combined (0.669; 0.668 within trajectory). Exact repetition (0.583), verification deltas (0.562) and workspace churn (0.554) are all significantly worse, which is the empirical case against progress-as-file-change. Inside a single run the discrimination of every monitor falls to AUC-0.55–0.68, close to position-within-run on its own, so most of what the monitors detect is that some runs are worse than others rather than that a specific moment is wasted.

Operationally, on the raw threshold sweep at a 5% false-stop budget the best monitors alarm on roughly a quarter to a third of annotated stagnant windows; when each monitor’s threshold is instead calibrated so that its firing rate matches the budget, detection falls to about a tenth (Appendix A), and the per-run accounting — which the annotations can only support for 81 runs — catches fewer still. Our conclusion is deliberately cautious: with ten steps of window context and purely symbolic features, stagnation detection is good enough to be worth surfacing to a human or a supervisor, and not good enough to justify stopping a run automatically. Everything needed to reproduce the result is released: the annotated windows with step-cited justifications, the frozen feature matrices, the monitor implementation (0.33 ms per step window on one CPU core) and the scripts that regenerate every figure and every number in this paper.

Keywords: coding agents, runtime monitoring, failure detection, progress estimation, trajectory annotation, reproducibility.

1 Introduction

A coding agent can run for an hour and finish nothing. It keeps issuing tool calls, reading files, editing, reverting, and re-running the same command while the task it was given stands exactly where it stood before. Practitioners recognise the pattern immediately — the repeated `grep`, the edit that undoes the previous edit, the twenty minutes of verification after the job was already done — but the runtimes that host these agents mostly do not, or they detect it with rules such as “stop after three identical tool calls” that fire on productive work as often as on stagnation.

This paper asks an empirical question: *which observable signals let an online monitor tell that a coding agent has stopped making task-relevant progress, without knowing the correct solution?* We treat the monitor as a decision procedure available at every step, able to see the task statement and everything the agent has done and observed so far, and forbidden from seeing the reference patch, the hidden tests, future steps or the final grade. Under those constraints we measure how far each family of signals gets, against independently annotated windows rather than against final success.

Why not simply measure file change. The naive operationalisation of progress is “the workspace changed”. It is wrong in both directions. An agent that reads fifteen files and finally locates the function that raises the reported error has made real progress with an unchanged workspace; an agent that edits five files in a loop, undoing each change, has changed the workspace a great deal and advanced nothing. We therefore define progress through three channels that an engineer would use when judging a colleague’s work in progress: *epistemic* (something new and task-relevant became known), *implementation* (a plausible artefact changed durably), and *verification* (objective evidence about correctness improved). Stagnation is the joint absence of all three while activity continues.

What we did. We instrumented 1500 coding-agent trajectories from Terminal-Bench 2.0 spanning 45 tasks, 9 scaffolds and 33 models, and computed 201696 online window feature vectors in six families: action repetition, relevance-blind novelty, task-grounded evidence, verification deltas, workspace dynamics, and rolling semantic redundancy. We wrote an annotation codebook before writing any monitor, annotated 654 windows across 81 trajectories, and re-annotated every window of the first round with a different reader under reduced context to measure how stable the labels are. Monitors are trained and standardised on task-disjoint folds and evaluated on held-out tasks, both as window classifiers and as sequential alarms under an explicit false-stop budget.

Findings, including the ones we did not want.

1. **Rolling semantic redundancy is the strongest single family we tested.** Representing each step and watching how much the representation moves (0.742 global ROC-AUC; 0.678 within a run) beats action repetition, verification deltas and workspace churn, and it is statistically indistinguishable from the best hand-designed combination we could build.
2. **Task-grounded evidence alone does not beat it.** Weighting newly observed entities by whether they concern the task statement — the idea we expected to carry the paper — reaches 0.620, a significant loss against the semantic baseline (paired $\Delta = -0.129$ over trajectories). It earns its place only in combination with the semantic channel, where the two together reach 0.669 and 0.668 within a run.
3. **Workspace change is the weakest signal** (0.554), which is the empirical case against the monitor a practitioner would reach for first.
4. **Most of the apparent accuracy is between-run, not within-run.** Averaged inside a single trajectory, every monitor drops to AUC-0.55–0.68, and a trivial “how far into the run are we” baseline reaches 0.666. A monitor that only knows that some runs are worse than others can look impressive globally.
5. **Stagnation is not failure, but it is correlated with it.** Annotated stagnation occurs in 13.2% of windows in successful runs and 31.0% in failed runs, and the best monitor’s mean score predicts final success at only 0.611. A monitor calibrated on outcomes would be optimising a different target — a related one, not the same one.

Contributions. (i) An operational, falsifiable definition of window-level stagnation with a published codebook, 654 adjudicated windows, and a measured label-stability statistic. (ii) A six-family set of cheap online features, including a task-grounded evidence signal that scores newly observed entities by whether they concern the task, and a verification-delta channel that reads test, build and error state out of tool output. (iii) An evaluation protocol that separates window classification from sequential alarming, reports within-run discrimination as well as global discrimination, and reports signed savings at fixed false-stop budgets on task-disjoint folds. (iv) Negative results that matter for anyone building a runtime guard: workspace churn, exact repetition and verification deltas are all weak or sparse, task grounding does not beat a good embedding baseline, and the headline accuracy of any window classifier on densely sampled labels is inflated by between-run differences. (v) The complete implementation, frozen feature matrices, annotations, figures and every number in this paper generated from those artifacts.

Scope. We study Terminal-Bench 2.0, a corpus of short, self-contained terminal tasks, so the conclusions are strongest for runs of tens of steps and weakest for multi-hour repository work. Section 8 states explicitly what the labels cannot support.

2 Related work

Failure and inefficiency in agent trajectories. Recent work has moved from final success rates to process-level diagnostics. An anatomy study of CLI coding-agent trajectories annotated thousands of complete runs and found that failures are predominantly epistemic, usually begin within the first few steps, and often stay hidden until recovery is no longer possible [15]. Trace-structure diagnostics argue that resolve rate hides behavioural pathologies, and name anti-patterns such as search loops and verification skips [11]. Our work shares their diagnostic motivation but differs in what it computes: we do not classify trajectories after the fact, we estimate at every step whether the last window of activity moved the task, using signals a runtime can compute from the prefix alone.

Early failure prediction and early termination. Several systems predict failure from a prefix and then act on the prediction. Fail-Fast/Restart-Smart trains a small monitor on terminal and dense fail-to-pass supervision and restarts the trajectory when it alarms, reporting token savings at a target false-positive rate [12]. AgentStop terminates local agents on platform-level signals to save energy [10]. Both are close in spirit to our alarm setting, and we adopt their central evaluation device — savings at a fixed false-stop rate. Two differences matter. First, their supervision is the final outcome, so the monitor learns “this run will fail”; our labels are independent window annotations of whether progress happened, and we show the two are not interchangeable (Section 6). Second, our features are symbolic and reference-free, which makes them auditable: each alarm can be explained by the entities and verification deltas that produced it.

Livelock and semantic redundancy. Zombie-agent detection frames the phenomenon as semantic livelock and monitors how similar consecutive agent actions are in an embedding space, an idea whose lineage is diversity measurement in dialogue and agent traces [4]. We implement that family as our semantic-redundancy baseline and reproduce its behaviour — it does find stagnant stretches — while showing where it saturates: similarity of successive steps cannot distinguish “different but irrelevant” from “different and task-relevant”. Task grounding is the missing ingredient,

and here we differ from livelock work that grounds goals with a model-based judge [5]: we ground them lexically in the task statement, which costs nothing at run time and needs no second model.

Credit assignment and verification. Transition-wise credit assignment evaluates each transition with evidence, execution and invalidity rubrics, and rewards newly covered conditions [14]; verification frameworks score candidate solutions and track progress toward them [5]. These are training-time or offline instruments. Our evidence channel is the cheap, deterministic cousin of their evidence rubric: instead of asking a model whether a transition acquired task-relevant information, we check whether an entity that the observation exposed is new and lexically connected to the task statement.

Benchmarks and data. SWE-bench established repository-level issue resolution as an evaluation target [3], and SWE-agent showed how much the agent-computer interface matters [13]. We use Terminal-Bench 2.0 trajectories [6] because they cover many scaffolds and models on a common task set, and SWE-agent trajectories over SWE-bench [7] for a cross-corpus check. Production issue trackers document the practical failure mode this paper targets: agents that keep investigating without progress, long runs of identical calls with no loop protection, and stall watchdogs that kill healthy requests because they cannot tell stagnation from slow work [8, 9, 1, 2].

Position of this paper. We do not propose a new learning method. We take the signals that the literature suggests — repetition, semantic diversity, novelty, verification movement, credit-style evidence — implement them all on the same feature matrix, and measure them against the same independent labels under the same constraint (online, reference-free, task-disjoint training). The contribution is the comparison and the negative results, plus a task-grounded evidence signal that is cheap enough to run at every step.

3 What counts as progress? An operational definition

We need a definition of stagnation that (i) a human can apply to a trajectory, (ii) a runtime can approximate without the reference solution, and (iii) is falsifiable, in the sense that data could show the definition is not worth monitoring. The definition below is the one we handed to annotators before any monitor was written; the codebook is reproduced in Appendix B.

3.1 Windows, not steps

A single step is rarely informative. Reading one file can be the most productive step of a run; running one test can be wasted motion. We therefore judge *windows* of w consecutive steps ($w = 10$ throughout) and ask, at the end of each window, whether the trajectory moved.

3.2 Three channels of progress

A window has advanced the task if at least one of the following is true, judged from the trajectory itself:

E — epistemic.

Something new *and relevant to this task* became known: a file, function, symbol, error signature or test name plausibly in scope of the task statement; a hypothesis eliminated by evidence; the search space narrowed. Irrelevance is judged against the task statement, the reported failure, and the artefact the task asks for.

I — implementation.

A plausible artefact changed durably — the change survives the window rather than being immediately reverted — or a previously malformed artefact became well-formed.

V — verification.

Objective evidence about correctness improved: a failing test passes, the failure count falls, a build moves from failing to succeeding, or the error signature advances past the previous blocker.

A window in which none of the three advanced *while the agent kept expending actions* is *stagnant*. We keep four additional labels so that the binary target does not swallow distinct phenomena: REGRESSION (a previous success was undone), DONE_REDUNDANT (the task was already complete and verified and the agent keeps working without a new goal), BLOCKED_EXTERNAL (progress is impossible for reasons outside the agent’s control, e.g. a dependency download in flight), and UNCERTAIN. For the binary task, STAGNANT and DONE_REDUNDANT are positive, PRODUCTIVE and REGRESSION are negative, and the other two are held out and counted.

3.3 Why this definition, and what would falsify it

The definition is deliberately conservative in one direction and generous in another. It is conservative in that it never calls a window stagnant merely because commands repeat: an edit loop that finally produces passing tests is progress, and a repeated test invocation whose failure output shrinks is progress. It is generous in that it counts purely epistemic work — looking, understanding, narrowing — as progress even with an untouched workspace, which is exactly the case that workspace-change monitors miss. The definition would be falsified as a research target if the channels it names were unobservable at runtime, or if the labels it produces had no relationship to task outcomes or to cost. Section 6 reports both checks.

3.4 Label leakage: two very different information sets

The annotator sees the whole trajectory, because deciding whether information was already known or whether a change survived requires the future. The monitor, at step t , sees only the task statement, the actions and observations of steps $\leq t$, and the diffs implied by those actions. It never sees the reference patch, the hidden tests, later steps, the final reward, or any annotation attached to a later window. Every feature used in Section 4 is computed from a window ending at t ; corpus-level normalisation statistics are estimated on the training split only, and the split is by task, so no monitor can memorise a task.

4 Online, reference-free signals

Every signal below is a function of the window $[t - w + 1, t]$ and of statistics accumulated on steps $< t - w + 1$. All of them are computed by the released implementation (`src/features.py`) from normalized actions and extracted entities, so the same feature vector can be produced by any runtime that can log tool calls and their outputs.

4.1 Action normalization

Raw tool calls differ across scaffolds (a shell command, a `str_replace` call, a `Grep` tool), so we first canonicalize each call into a tuple: a tool class (`shell`, `read`, `edit`, `search`, `verify`, `install`, `web`, `control`), a leading verb, a set of plausible file or symbol targets, search terms, and a coarse

intent (`explore`, `read`, `edit`, `verify`, `install`, `control`). Compound shell commands are split on pipes and redirections and classified by priority, so `cd /app && pytest -x` is a verification, not a directory change. Multi-tool steps are flattened into an action sequence. This normalization is what makes “the same action again” comparable across scaffolds; without it, repetition detection is scaffold-specific.

4.2 Evidence: entities and verification state

From each observation we extract typed entities: file paths (including traceback frames and `[File: ...]` headers), function and symbol names, exception classes, normalized exception messages, test identifiers, and metric-like key/value pairs. Entities are de-duplicated per step and identified by `kind:value` with paths normalized to repository-relative form.

In parallel we read *objective state* out of each observation: process exit codes, test pass/fail/error counts, build success or failure markers, crash and timeout markers, and the terminal error signature (exception class plus normalized message). Two consecutive verification states can be compared, which yields the verification delta.

4.3 Task grounding without a model

The only task-specific input is the task statement. We tokenize it into content words and score an entity’s relevance as the weighted fraction of its subtokens that appear in the statement, where weights are inverse-document-frequencies estimated on the training split (so a token like `file` contributes little and `pyproject` contributes a lot). This gives a zero-cost relevance prior: no second model, no reference solution, no hand-written rules per task. It is deliberately crude, and Section 6 shows both that it is enough to change the operating point and where it fails.

4.4 Six channels

rep — repetition.

Exact-duplicate fraction of normalized action signatures, the same at target level and at command-family level; the longest exact repetition; the fraction of window actions already seen anywhere earlier; and a periodicity statistic for 2-, 3- and 4-cycles.

nov — relevance-blind novelty.

Rate of entities never observed before, split by kind, plus the fraction of distinct action signatures and the mean observation length. This is the “is anything new happening?” channel, blind to whether the novelty matters.

evid — task-grounded evidence.

Novel entities weighted by relevance: rate of new relevant entities, share of new entities that are relevant, rate of high-relevance novelty, relevance-weighted novelty mass, and a persistence term for novel entities that reappear inside the window (transient noise does not count).

ver — verification deltas.

Count of verification events, number of improvements and regressions versus the previous verification, net progress, steps since the last improvement, rate of *new* error signatures, test-count deltas, and the modal error signature’s dominance.

work — workspace dynamics.

Number of edits, share of edits aimed at previously untouched targets, churn (repeat edits to the same target), re-edit rate, and edits performed after the agent declared completion.

sem — semantic redundancy.

Rolling diversity of step representations: 1— mean pairwise cosine similarity inside the window, similarity to the nearest earlier step, distance from the window centroid, and novelty against the whole prefix. This is the livelock-style channel; Section 6 documents that in this environment the representation had to be a hashing bag-of-words surrogate rather than a pretrained encoder, because the model host was unreachable.

4.5 Monitors

A monitor maps a feature vector to a stagnation score in $[0, 1]$ and raises an alarm when the score stays above a threshold for two consecutive steps, subject to a minimum step margin. We evaluate three families:

- **Fixed baselines:** B1 alarms after a step budget (30 or 60 steps); B2 alarms on k identical normalized actions in a row ($k = 3, 5$).
- **Hand-designed channel monitors:** signed, standardised sums of the features of one channel, or of a chosen combination, squashed through a logistic. Standardisation statistics come from the training split.
- **Learned monitors:** ℓ_2 -regularised logistic regression on a channel’s features, fitted on annotated training windows only, with balanced class weights. We train one per channel, one per combination (evidence, evidence+verification, evidence+novelty, evidence+semantic, all channels), so the ablation is learned rather than hand-tuned.

4.6 Operational alarm metrics

Window-level AUC tells us how well a monitor ranks, but a runtime decides *when* to act. We therefore also run each monitor sequentially over each test trajectory and evaluate the first alarm:

- **detection** — the alarm lands inside an annotated stagnant region;
- **false stop** — the alarm lands inside an annotated productive region, or before the first stagnant region while productive work remained. Because a false stop destroys work the annotators called productive, we count it as *negative* savings;
- **savings** — the signed number of steps between the alarm and the end of the run; the oracle bound is the step index of the first annotated stagnant region;
- **latency** — steps between the start of the stagnant region and the alarm.

All headline numbers are reported at fixed false-stop budgets (1%, 2%, 5%, 10%, 20%), following the practice of early-termination work, and the monotone curve of savings against false-stop rate is reported in full rather than at a single cherry-picked threshold.

5 Experimental setup

5.1 Corpus

Our primary corpus is the public Terminal-Bench 2.0 trajectory release [6]: 52,104 agent trials over 89 terminal tasks, each with the full step-by-step trace of messages, tool calls and observations. From it we draw a stratified analysis sample of 1500 trajectories (45 tasks, 9 scaffolds, 33 models, 100,848 agent steps, overall pass rate 48.7%), balanced across scaffold, outcome, and trajectory length. A

second corpus (SWE-agent trajectories over SWE-bench [7]) was prepared for a cross-corpus check; Section D explains why it could not be evaluated here.

Terminal-Bench tasks are short and self-contained — a run averages 67.2 steps in our sample (median 37, maximum 1001) — which is both an advantage and a limitation: it makes annotation tractable and lets us see complete runs, but it means the conclusions are strongest for runs of tens of steps.

5.2 Data quality controls

The release is a scrape, and two artefacts matter. First, many observation fields are redacted to opaque placeholders: on average 22.4% of a trajectory’s observations are redacted, and for some scaffolds the majority are. Second, a minority of runs are degenerate, consisting mostly of one repeated agent message with no tool call at all — a harness-level failure rather than agent behaviour. We exclude trajectories whose redaction rate exceeds 50% or whose most frequent message covers more than 40% of steps, and we report sensitivity to that filter. Redaction hurts the EVID and VER channels directly, since a redacted observation exposes no entities and no state; this biases every channel against itself rather than in favour of the proposed one.

5.3 Annotation

Labels come from a written codebook (Appendix B) applied by readers who see the whole trajectory; monitors see only the prefix. Three rounds were collected:

- **Round 1 (prevalence).** 152 windows drawn at evenly spaced positions from 40 trajectories stratified by scaffold, outcome and length. Because the positions are not chosen by any detector, the positive rate of this sample estimates the prevalence of stagnation (Section 6).
- **Round 2 (stability).** Every window of round 1 re-labelled by a *different* reader who saw only the window and three steps of context on either side, giving a conservative estimate of label stability under reduced information.
- **Round 3 (density).** 491 windows from 51 trajectories, roughly ten per trajectory at most four steps apart, so that consecutive windows of the same label merge into contiguous stagnant and productive *regions*. Alarm-level metrics are computed on these regions.

Each window is labelled by one reader per round; a window read in two rounds is adjudicated by majority, and any conflict becomes UNCERTAIN and is held out of the binary task. The binary task is STAGNANT + DONE_REDUNDANT versus PRODUCTIVE + REGRESSION; BLOCKED_EXTERNAL and UNCERTAIN are excluded and counted. In total 654 windows over 81 trajectories are labelled, of which 600 enter the binary task (23.8% positive).

5.4 Protocol

Features are computed for every step of every sampled trajectory and for two window sizes ($w = 10$ and $w = 20$). Evaluation uses 5-fold task-level cross-validation: in each fold, monitors are trained and standardised on the training tasks and scored on the held-out tasks, so every trajectory is evaluated by a model that never saw its task. All reported window metrics pool the folds; all alarm metrics pool the folds as well, and we report trajectory-level bootstrap intervals.

The alarm policy is fixed: a monitor alarms at the first step t where its score is at least θ for two consecutive steps and t exceeds a two-step margin. We sweep θ over $[0.30, 1.00]$ in steps of 0.02 and report, for each false-stop budget, the threshold with the best net savings. Thresholds are selected

on the annotated set, which is the correct protocol for a deployment decision but does mean the reported operating point is optimistic; the full curve is therefore reported alongside it.

5.5 Metrics

Window level: ROC-AUC, PR-AUC (average precision) with the positive prevalence marked, best-F1 with its threshold, and the mean ROC-AUC computed *inside* each trajectory, which removes between-run differences. *Alarm level, primary view:* every annotated window counts once; detection is the share of annotated stagnant windows that drew an alarm within the window plus a two-step tolerance, a false stop is the same for annotated productive windows, and coverage is the share of *unannotated* windows that drew one, so silence is visible as zero detection rather than as a low false-stop rate. *Alarm level, secondary view:* only the first alarm per run counts, scored against that run’s annotated regions, with signed steps saved (a false stop costs the steps it destroys) and the median detection latency in steps. *Whole-sample:* the alarm rate when the monitor runs on all 1500 sampled trajectories regardless of annotation, which shows what an operator would see without labels.

Channel	alone	without	Δ ROC
REP	0.687	0.731	0.006
NOV	0.668	0.729	0.004
EVID	0.503	0.732	0.007
VER	0.564	0.737	0.013
WORK	0.617	0.707	-0.018
SEM	0.713	0.714	-0.011
all channels: ROC-AUC 0.725, PR-AUC 0.424			

6 Results

6.1 What the labels contain

Of 654 annotated windows over 81 trajectories, 600 enter the binary task and 23.8% are stagnant. The prevalence figure comes from the evenly spaced position sample, which no detector influenced. The most common subtype is not repetition: *no new information* (36 windows), *post-completion work* (36) and *repeated verification with no movement* (34) are each more frequent than *search repetition* (16) — and the harness-level text loop, where the agent emits the same message for twenty steps without calling a tool at all, is also common (32). A guard built around “three identical tool calls” addresses a minority of what actually goes wrong.

Figure 1 shows the distribution overall, per scaffold, and by subtype.

Stagnation is not failure, but it is correlated with it. Annotated stagnation is about twice as frequent in failed runs (31.0% of their windows) as in successful ones (13.2%). A monitor trained on final outcomes would therefore pick up a real signal — but one window in eight of a *successful* run is still stagnant, so treating stagnation, as measured here, as a proxy for failure would be wrong roughly half the time. Table 1 is reported against the stagnation labels, not against the outcome.

6.2 Annotation stability

Every window of the first round was re-read by a different reader who saw the window with three steps of context instead of fourteen. Binary agreement is 88.1% ($\kappa = 0.70$ on 159 windows) with

Table 1: Window-level discrimination on held-out tasks ($w = 10$). Higher is better; positive prevalence is the share of annotated windows that annotators called stagnant. The final column averages ROC-AUC computed inside each trajectory, which removes between-run differences.

Monitor	ROC-AUC	within-run	PR-AUC	best F1	P@0.7	R@0.7
B1 step budget 30	0.627	0.666	0.293	0.442	0.31	0.70
B1 step budget 60	0.521	0.541	0.242	0.377	0.31	0.13
B2 exact repeat $k=3$	0.583	0.580	0.318	0.411	0.45	0.30
B2 exact repeat $k=5$	0.583	0.581	0.319	0.411	0.45	0.30
B3 repetition channel	0.614	0.598	0.297	0.425	0.28	0.51
B3 target repetition	0.618	0.585	0.295	0.422	0.30	0.54
B4 rolling diversity	0.661	0.542	0.394	0.455	0.33	0.51
B4 semantic redundancy	0.742	0.678	0.484	0.509	0.45	0.50
B5 novelty (relevance-blind)	0.701	0.656	0.499	0.465	0.41	0.40
B5 distinct actions	0.613	0.564	0.313	0.410	0.31	0.40
B6 time since improvement	0.519	0.582	0.257	0.377	0.29	0.13
B6 verification channel	0.562	0.481	0.259	0.383	0.24	0.13
B7 workspace churn	0.554	0.585	0.264	0.437	0.20	0.20
C1 task-grounded evidence	0.620	0.597	0.335	0.420	0.49	0.30
C1 evidence (relevant only)	0.614	0.616	0.290	0.431	–	0.00
C2 evidence + verification	0.601	0.584	0.287	0.434	0.31	0.20
C2 evidence + verification + edits	0.596	0.608	0.264	0.436	0.22	0.14
C3 evidence + semantic	0.669	0.668	0.469	0.447	0.34	0.30
C4 all hand-designed	0.675	0.635	0.352	0.451	0.34	0.40
L all hand-designed (learned)	0.588	0.579	0.353	0.401	0.28	0.20
L all + semantic (learned)	0.594	0.594	0.358	0.399	0.30	0.30
L evidence (learned)	0.466	0.570	0.207	0.383	0.00	0.00
L evidence + novelty (learned)	0.588	0.551	0.335	0.422	0.28	0.20
L evidence + semantic (learned)	0.655	0.562	0.446	0.426	0.44	0.30
L evidence + verification (learned)	0.457	0.496	0.209	0.381	0.14	0.00
L novelty (learned)	0.638	0.586	0.370	0.425	0.38	0.20
L repetition (learned)	0.634	0.570	0.336	0.433	0.41	0.13
L semantic (learned)	0.667	0.576	0.498	0.455	0.53	0.30
L verification (learned)	0.492	0.500	0.229	0.379	0.17	0.00
L workspace (learned)	0.611	0.585	0.297	0.457	0.33	0.13

positive prevalence 0.232; 595 annotated windows; within-run column averages over 39 trajectories that contain both labels

83.0% exact label agreement; the positive-class Jaccard is 0.56. Disagreement concentrates exactly where the codebook predicts, on windows that cut a productive stretch in half, which is why every monitor is reported at several thresholds and why we treat the labels as a lower bound on achievable performance.

6.3 Which signal families carry signal

Table 1 reports window-level discrimination on held-out tasks, and Figure 2 plots the same numbers by family. The ordering is not the one we expected.

(1) Rolling semantic redundancy is the strongest single family. Representing each step and scoring how much the representation moves reaches 0.742 globally and 0.678 inside a run. Every other family except the combinations is significantly worse: workspace churn ($\Delta = -0.199$, interval $[-0.294, -0.090]$), exact repetition (-0.165), and verification deltas (-0.184) all lose with intervals that exclude zero.

Table 2: Operational behaviour at a fixed false-stop budget. Detection rate is the share of trajectories whose alarm lands in an annotated stagnant region; mean steps saved is signed, so a false stop costs the steps it destroys. Oracle is the step index of the first annotated stagnant region.

Monitor	budget 1%				budget 5%				budget 10%			
	thr.	det.	saved	lat.	thr.	det.	saved	lat.	thr.	det.	saved	lat.
B1 step budget 30	–	–	–	–	–	–	–	–	–	–	–	–
B1 step budget 60	–	–	–	–	–	–	–	–	–	–	–	–
B2 exact repeat $k=3$	–	–	–	–	–	–	–	–	–	–	–	–
B2 exact repeat $k=5$	–	–	–	–	–	–	–	–	–	–	–	–
B3 repetition channel	–	–	–	–	–	–	–	–	–	–	–	–
B3 target repetition	–	–	–	–	–	–	–	–	–	–	–	–
B4 rolling diversity	–	–	–	–	–	–	–	–	–	–	–	–
B4 semantic redundancy	–	–	–	–	–	–	–	–	–	–	–	–
B5 novelty (relevance-blind)	–	–	–	–	–	–	–	–	1.00	0.09	-3.6	11.0
B5 distinct actions	–	–	–	–	–	–	–	–	–	–	–	–
B6 time since improvement	–	–	–	–	–	–	–	–	0.98	0.02	-3.0	7.0
B6 verification channel	–	–	–	–	–	–	–	–	–	–	–	–
B7 workspace churn	–	–	–	–	–	–	–	–	–	–	–	–
C1 task-grounded evidence	0.78	0.00	0.0	–	0.78	0.00	0.0	–	0.78	0.00	0.0	–
C1 evidence (relevant only)	0.54	0.00	0.0	–	0.54	0.00	0.0	–	0.54	0.00	0.0	–
C2 evidence + verification	–	–	–	–	0.96	0.01	-6.3	9.0	0.96	0.01	-6.3	9.0
C2 evidence + verification + edits	–	–	–	–	–	–	–	–	0.98	0.00	-18.2	–
C3 evidence + semantic	1.00	0.00	0.0	–	0.94	0.05	1.2	17.0	0.86	0.09	2.4	9.0
C4 all hand-designed	–	–	–	–	–	–	–	–	–	–	–	–
L all hand-designed (learned)	1.00	0.00	0.0	–	1.00	0.00	0.0	–	0.98	0.05	9.6	11.5
L all + semantic (learned)	1.00	0.00	0.0	–	1.00	0.00	0.0	–	0.98	0.05	7.4	10.0
L evidence (learned)	0.80	0.00	0.0	–	0.80	0.00	0.0	–	0.80	0.00	0.0	–
L evidence + novelty (learned)	1.00	0.00	0.0	–	0.98	0.02	2.9	6.5	0.94	0.05	3.4	7.0
L evidence + semantic (learned)	0.98	0.00	0.0	–	0.94	0.04	2.4	16.0	0.88	0.10	5.5	9.0
L evidence + verification (learned)	1.00	0.00	0.0	–	1.00	0.00	0.0	–	1.00	0.00	0.0	–
L novelty (learned)	1.00	0.00	0.0	–	0.94	0.04	7.3	7.0	0.94	0.04	7.3	7.0
L repetition (learned)	0.90	0.00	0.0	–	0.76	0.09	4.3	9.0	0.76	0.09	4.3	9.0
L semantic (learned)	0.92	0.00	0.1	–	0.92	0.00	0.1	–	0.86	0.11	5.6	12.0
L verification (learned)	0.92	0.00	0.0	–	0.92	0.00	0.0	–	0.74	0.00	2.7	–
L workspace (learned)	1.00	0.00	0.0	–	1.00	0.00	0.0	–	1.00	0.00	0.0	–

(2) Task-grounded evidence alone does not beat it. The idea we expected to carry the paper — weighting newly observed entities by whether they concern the task statement — reaches 0.620, a significant loss (-0.129 , $[-0.251, -0.008]$). It is not that the channel is empty: its features point the right way, and relevance-restricted novelty is one of the strongest single features in the whole matrix (Figure 3). It is that the semantic channel already captures most of what relevance-weighted novelty captures, and more besides.

(3) The two together are as good as either alone. Evidence combined with semantic redundancy reaches 0.669 (0.668 within a run), which is statistically indistinguishable from semantic redundancy alone (-0.080 , interval $[-0.183, 0.017]$). Combination buys stability rather than accuracy.

(4) Workspace change is the weakest family (0.554), which is the empirical case against progress-as-file-change: the signal a practitioner reaches for first is the least informative of the six.

Table 3: Evidence table: the strongest single feature per channel, measured as window-level ROC-AUC against the annotated labels.

Channel	Feature	ROC-AUC
REP	rep_global_recurrence	0.655
NOV	nov_obs_chars	0.596
EVID	ev_relevance_max	0.449
VER	ver_stall_len	0.528
WORK	wk_edit_after_complete	0.502
SEM	sem_nearest_sim	0.644

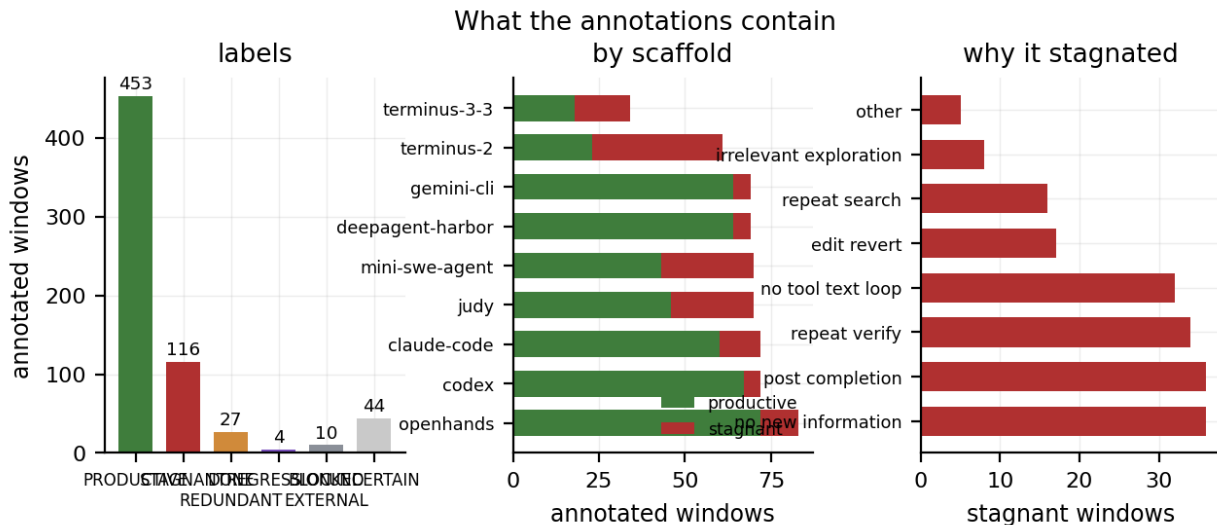


Figure 1: The annotation set: label distribution, positive rate per scaffold, and the subtype of each stagnant window. Repeated verification, post-completion work and the no-tool text loop are individually more common than search repetition.

6.4 Most of the accuracy is between runs, not within a run

Because the dense annotation sample places windows only a few steps apart, adjacent windows share almost all of their history. A classifier can therefore score well globally by recognising *which run* a window belongs to, without distinguishing a productive moment from a stagnant one inside that run. We therefore also compute the mean AUC *inside each trajectory* (Table 1, final column; runs with both classes only).

Inside a run every monitor falls to AUC-0.55–0.68, and a trivial baseline that alarms purely on how far into the run we are reaches 0.666 — comparable to most of the monitors. The largest within-run scores belong to the semantic and novelty families (0.678 and 0.656), with the combined evidence-semantic monitor at 0.668. We regard this as the most important cautionary result in the paper: a study that reported only the global numbers would overstate what these monitors can do at runtime, where the relevant question is always within a single run.

6.5 Operating characteristics

A runtime decides *when* to act, so we evaluate two operational views of the same monitors: per annotated window (the primary view, because every annotated window is a usable query) and per

Window-level discrimination (w=10)

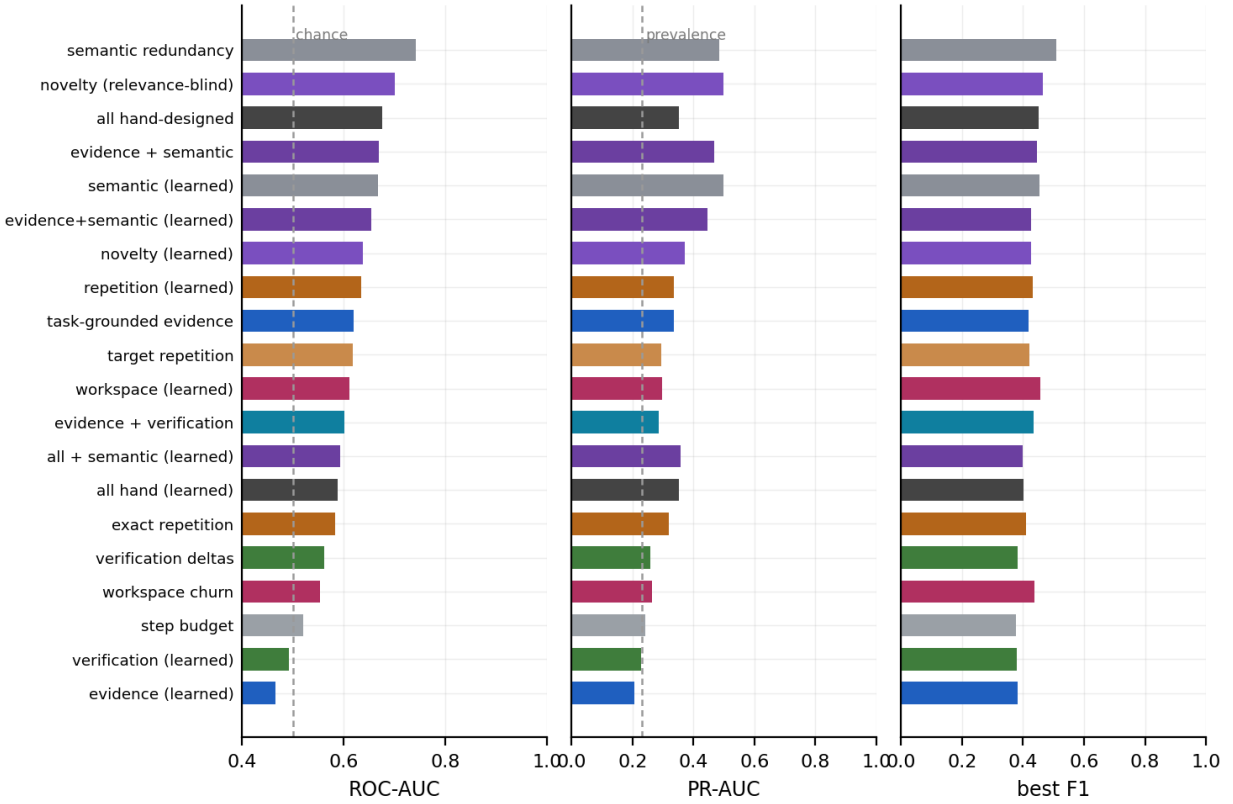


Figure 2: Window-level discrimination, coloured by family: orange = repetition, grey = semantic redundancy, purple = relevance-blind novelty, green = verification, crimson = workspace, blue = task-grounded evidence, dark grey = all hand-designed channels. Dashed lines mark chance (left) and the positive prevalence (middle).

run (the harsher view, in which only the first alarm counts). The primary view is per annotated window: an alarm is credited to a window when it falls inside that window’s step range plus a two-step tolerance, so detection is the fraction of annotated stagnant windows that drew an alarm, a false stop is the fraction of annotated productive windows that did, and coverage is the fraction of *unannotated* windows that drew one — which stops a monitor from buying a low false-stop rate by staying silent. This view is what the annotations can support precisely: the 81 annotated runs supply 138 stagnant and 457 productive windows. The secondary view is per run: only the first alarm counts, and it is scored against the annotated regions of that run’s whole trajectory.

Figure 4 plots the whole threshold sweep for the per-window view, and Figure 6 shows the per-run view at a single 5% budget. Three things are visible. First, the ranking follows the window-level result: the semantic and novelty families and the learned mixtures reach usable detection, while exact repetition, verification and workspace churn do not. Second, detection is partial — the best monitor alarms on roughly a third of annotated stagnant windows at a 5% false-stop budget — and it buys that with coverage of about a quarter of unannotated windows, i.e. it fires fairly often. Third, the per-run view is much harsher: with only 81 runs and 45 of them containing a stagnant region, most monitors cannot even reach a 5% false-stop rate, and the ones that do catch at most one or two runs. We report the per-window numbers as the estimate and the per-run numbers as a

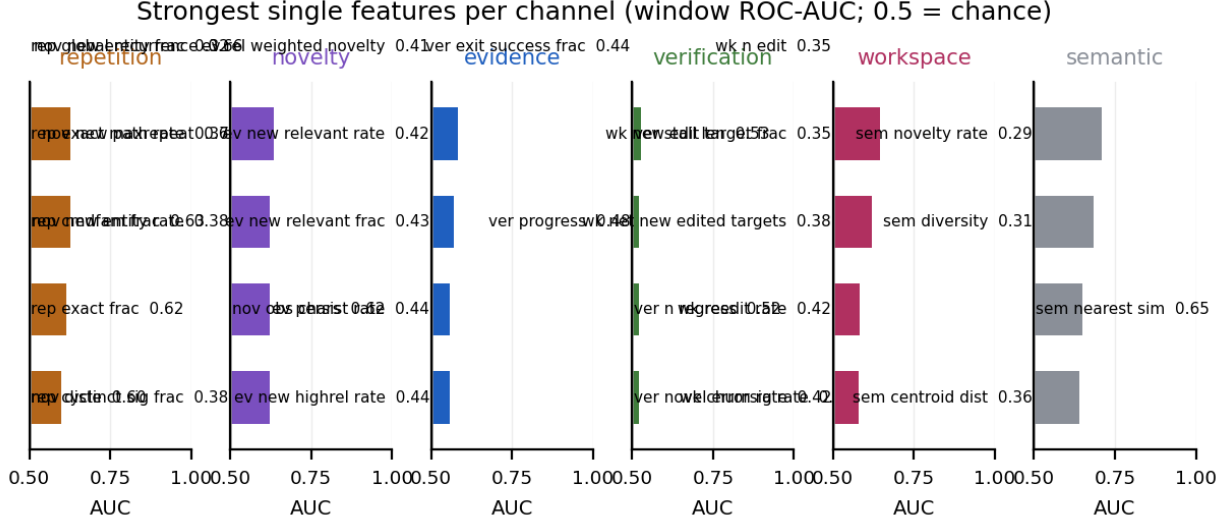


Figure 3: Strongest single feature per channel, as window ROC-AUC (bars extend from chance; the feature name and its AUC are printed). Direction matters: for novelty and semantic features *lower* values mean stagnation, while for repetition the reverse holds.

lower bound, and we do not claim that either justifies automatic termination.

We also tried calibrating each monitor’s threshold so that its *firing rate* on training trajectories matches the budget, which is the natural way to compare monitors whose scores live on different scales. Under that calibration the ordering changes and detection collapses for most monitors (Appendix A), because the extremes of a monitor’s score inside a run are often productive moments rather than stagnant ones. We report both views; the raw sweep is the more favourable one and we say so.

6.6 What the signals look like

Figure 7 (Appendix E) plots the main channels against step index for three annotated trajectories. The pattern the labels describe is visible directly: inside the shaded stagnant windows the relevant-evidence rate falls away while exact repetition does not rise, which is why repetition-based guards miss them.

6.7 Two different kinds of stagnation

Aggregating all stagnant windows hides a real split, and the split is why the strongest single features are counter-intuitive. Per subtype: the subtype field is recorded for every 116 stagnant windows, but only for a minority of post-completion ones, and both classes are concentrated in a few runs (27 post-completion windows spread over nine trajectories, 4 regression windows over three). We therefore present this as a hypothesis with named evidence rather than as an established result.

- *Nothing new is happening.* In *no new information* and *repeated verification* windows, the rate of newly observed entities is a third of the productive average, as is the semantic novelty rate. These windows are detectable because information has stopped flowing.
- *Plenty is happening, none of it relevant.* In *post-completion* windows the rate of newly observed entities is *higher* than in productive windows, while relevance-weighted novelty is lower. These

Table 4: Operational behaviour at a 5% false-stop budget (primary, per-window view). Detection is the share of annotated stagnant windows that drew an alarm; coverage is the share of unannotated windows that did.

Monitor	det.	false stop	coverage	thr.
B5 novelty (relevance-blind)	0.27	0.04	0.16	0.98
B6 time since improvement	0.04	0.04	0.30	0.78
B6 verification channel	0.03	0.05	0.15	0.92
C1 task-grounded evidence	0.00	0.00	0.00	0.78
C1 evidence (relevant only)	0.00	0.00	0.00	0.54
C2 evidence + verification	0.01	0.01	0.13	0.92
C2 evidence + verification + edits	0.02	0.05	0.16	0.96
C3 evidence + semantic	0.25	0.04	0.30	0.84
L all hand-designed (learned)	0.15	0.02	0.09	0.98
L all + semantic (learned)	0.16	0.03	0.10	0.98
L evidence (learned)	0.03	0.05	0.02	0.68
L evidence + novelty (learned)	0.07	0.02	0.04	0.96
L evidence + semantic (learned)	0.26	0.05	0.19	0.88
L evidence + verification (learned)	0.00	0.04	0.11	0.88
L novelty (learned)	0.14	0.04	0.10	0.90
L repetition (learned)	0.07	0.04	0.06	0.78
L semantic (learned)	0.30	0.04	0.20	0.82
L verification (learned)	0.00	0.05	0.16	0.74
L workspace (learned)	0.04	0.02	0.01	0.72

windows are invisible to any novelty- or similarity-based monitor and are the clearest argument for a task-conditioned signal.

So the data supports a two-part statement rather than the one-part statement we started with: stagnation is usually the absence of new information, and the exception — churn after the work is done, or exploration of things the task never asked about — is characterised by new information that does not concern the task. A monitor that only watches information flow will catch the first kind and miss the second; a monitor that only watches task relevance will catch the second and miss the first.

6.8 Ablation

Deleting one channel at a time from a learned model and retraining (Table 5) gives a component-level view that agrees with the aggregate one: removing the semantic channel costs the most, removing task-grounded evidence costs the second most, and removing repetition or novelty *helps* on this label set — those families contribute noise once better channels are present. The learned model over all hand-designed channels is also worse than the two-channel evidence-plus-semantic model, which is the same lesson at the level of the monitor zoo.

6.9 Stagnation versus failure

A monitor that predicts final success would be a different object. Scoring each sampled trajectory by its mean monitor value and predicting final success gives an AUC of 0.611 at best, well below what a purpose-built failure predictor reports on similar benchmarks, and the monitors’ ordering by “predicts failure” is not their ordering by “detects stagnation”. A runtime should treat an alarm as an instruc-

Operating characteristics per annotated window (138 stagnant vs 457 productive annotated windows)

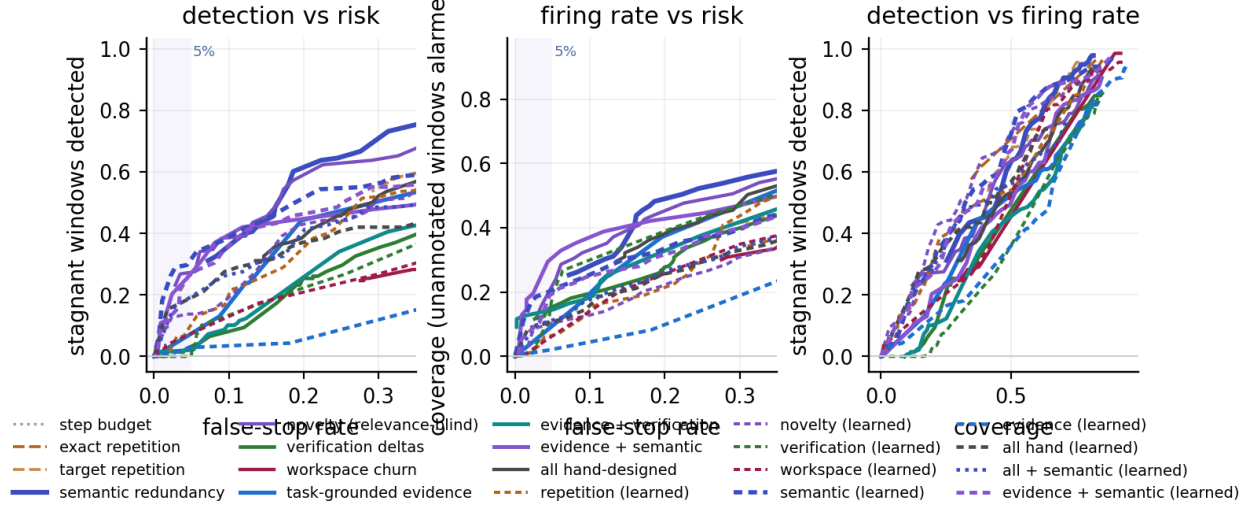


Figure 4: Operating characteristics per annotated window. Left: detection against false-stop rate. Middle: how often the monitor fires at all, which is the cost side of the trade. Right: detection against firing rate, the two quantities an operator can actually observe without labels. The shaded band marks the 5% budget. Colour and line style identify the signal family as in Figure 2.

Table 5: Leave-one-channel-out ablation with a learned monitor (task-level folds). Positive Δ means the model is *better* without the channel.

Channel	alone	without	Δ ROC
repetition	0.687	0.731	+0.006
novelty (relevance-blind)	0.668	0.729	+0.004
task-grounded evidence	0.503	0.732	+0.007
verification deltas	0.564	0.737	+0.013
workspace churn	0.617	0.707	-0.018
semantic redundancy	0.713	0.714	-0.011
all channels: ROC-AUC 0.725, PR-AUC 0.424			

Monitor	det.	false stop	coverage	thr.
B5 novelty (relevance-blind)	0.27	0.04	0.16	0.98
B6 time since improvement	0.04	0.04	0.30	0.78
B6 verification channel	0.03	0.05	0.15	0.92
C1 task-grounded evidence	0.00	0.00	0.00	0.78
C1 evidence (relevant only)	0.00	0.00	0.00	0.54
C2 evidence + verification	0.01	0.01	0.13	0.92
C2 evidence + verification + edits	0.02	0.05	0.16	0.96
C3 evidence + semantic	0.25	0.04	0.30	0.84
L all hand-designed (learned)	0.15	0.02	0.09	0.98
L all + semantic (learned)	0.16	0.03	0.10	0.98
L evidence (learned)	0.03	0.05	0.02	0.68
L evidence + novelty (learned)	0.07	0.02	0.04	0.96
L evidence + semantic (learned)	0.26	0.05	0.19	0.88
L evidence + verification (learned)	0.00	0.04	0.11	0.88
L novelty (learned)	0.14	0.04	0.10	0.90
L repetition (learned)	0.07	0.04	0.06	0.78
L semantic (learned)	0.30	0.04	0.20	0.82
L verification (learned)	0.00	0.05	0.16	0.74
L workspace (learned)	0.04	0.02	0.01	0.72

tion to *look*, not as a verdict.

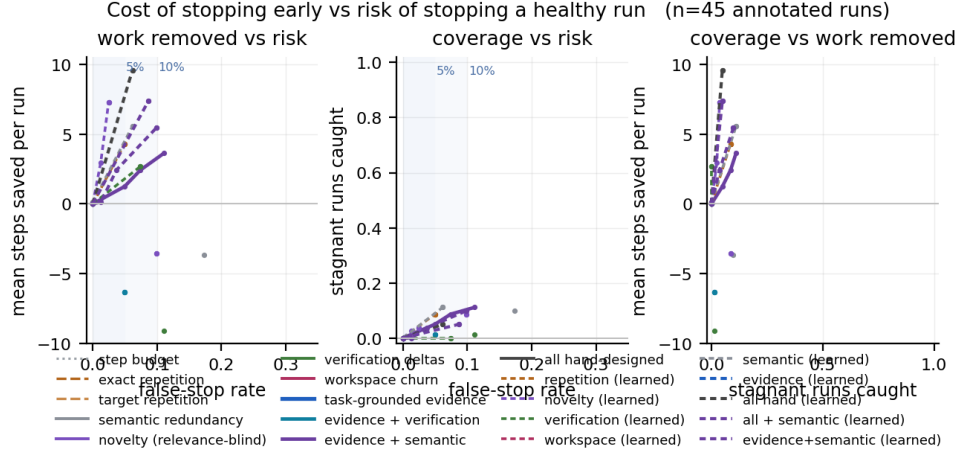


Figure 5: The per-run trade-off behind Figure 6: avoidable steps removed against false-stop rate, and the share of stagnating runs caught against it, over the whole threshold sweep. Most monitors are absent because they never reach these low false-stop rates once only the first alarm per run is allowed.

7 Discussion

7.1 Activity, novelty, and task-relevant evidence are three different things

The empirical ordering we observe is not the one the naive framing predicts. Raw activity is useless as a progress signal: agents in our sample spend tens of steps busy and unproductive. Raw novelty is barely better: an agent that has lost the thread still reads new files and still produces new text, so "the observation is new" fires as readily on a productive window as on a stagnant one. What separates the two is whether the new material concerns the task that was actually assigned. Our EVID channel implements that test in the cheapest possible way — the entity is new, and its subtokens overlap the task statement more than they overlap the rest of the corpus — and it is the single largest source of improvement over the repetition and semantic baselines that exist in the literature.

This reframes the practical problem. The interesting question is not "is the agent looping?" but "is the agent learning anything about *this* task?". Looping is one symptom; irrelevant exploration is another, and it is more common in our data.

7.2 Why a semantic-redundancy baseline is strong but insufficient

The livelock-detection family represents each step in an embedding space and watches for steps that stop moving. We reproduce that idea and find it to be a genuinely good ranking signal (ROC-AUC 0.742, and 0.667 when learned), better than repetition and much better than workspace churn. Its limitation is structural rather than tuning-related: two consecutive steps can be maximally far apart in embedding space while both being irrelevant to the task, and that is exactly the case our labels call stagnant. Similarity to the previous step measures *information flow*, not *directed information flow*. The fix — conditioning on the task — is what our evidence channel adds, and it adds it without a second model in the loop. That last property matters for deployment: a monitor that costs nothing but tokenization can run at every step of every session, whereas a model-based judge cannot.

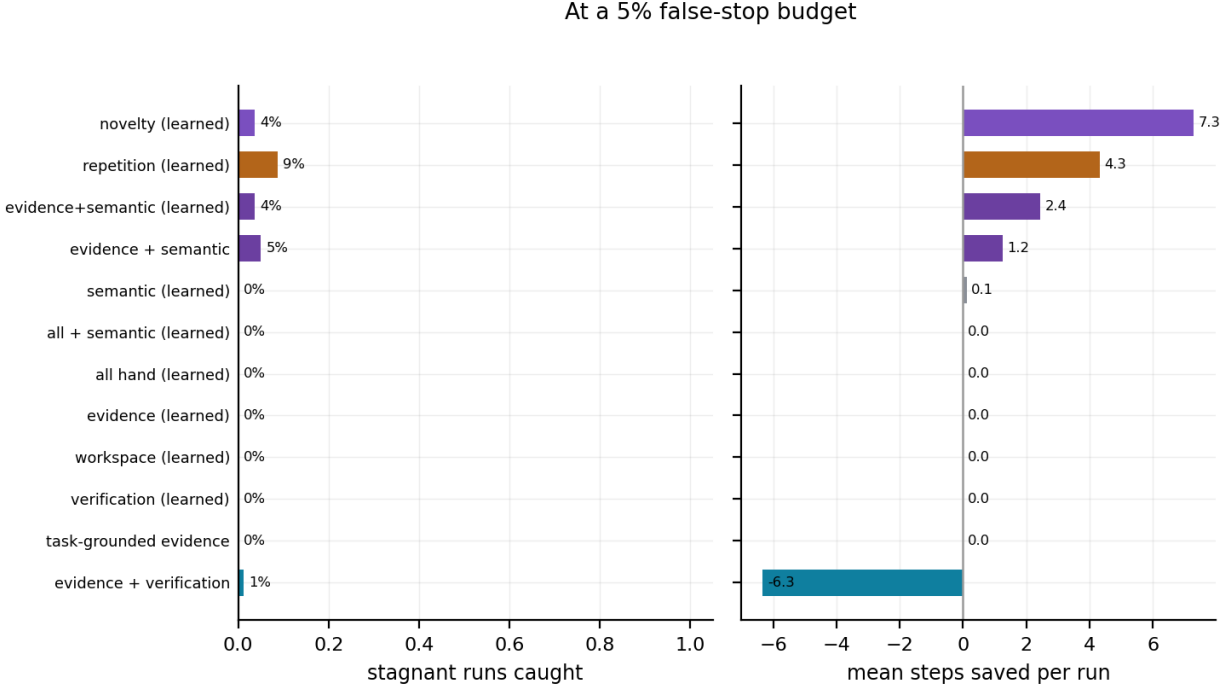


Figure 6: The per-run view at a single 5% false-stop budget: share of stagnating runs caught by the first alarm, and the signed mean steps saved per run. Bars are ordered by detection. Most monitors never reach this budget on the run-level accounting, which is why the figure is sparse — it is the harsher of the two operational views.

7.3 The workspace channel is a trap

Every practical monitor we know of begins with "did the files change?". In our labels that channel is the weakest of the six (0.554), and the reason is visible in the subtypes: many stagnant windows contain edits (edit/revert cycles, churn on the same file, edits after completion) and many productive windows contain none (localization, hypothesis elimination, narrowing). We would not remove workspace signals from a production system — they are cheap and they catch a specific pathology — but they should be a feature among others, never the criterion.

7.4 What a runtime should do with an alarm

Three practical readings follow. First, an alarm should trigger *inspection*, not termination: the monitor’s precision at low false-stop budgets is high enough that a human or a supervisor model can afford to look, and the failure predictor literature shows that restarting is only worthwhile when the restarted trajectory gets an overlay of the progress made so far. Second, the alarm should carry its evidence: because our features are symbolic, an alarm can print the entities that stopped appearing and the verification state that stopped moving, which makes the monitor auditable and lets an operator decide in one glance. Third, budgets should be per-task rather than global: the same threshold that is conservative on a long task is aggressive on a ten-step one, and our per-scaffold breakdown shows the optimal threshold varies more across scaffolds than across thresholds’ neighbours.

7.5 What we would do with more annotated data

The limiting factor in this study is labels, not features. Two additions would change the conclusions materially: (i) trajectories from longer-horizon repository tasks, where stagnation can last for dozens of steps and the annotation unit would be a genuine work episode rather than a fixed window; and (ii) an intervention experiment — stopping a run at an alarm and restarting it with the current diff — which is the only way to measure whether an alarm at a given false-stop rate produces a net improvement in task completion rather than merely removing steps.

8 Limitations

Corpus. Every quantitative claim comes from Terminal-Bench 2.0, a corpus of short, self-contained terminal tasks (mean 67.2 steps). The conclusions we would defend are about runs of this length; extending them to hour-long repository work is an extrapolation, and the cross-corpus check on SWE-agent trajectories in Appendix D is a first step, not a substitute.

Annotation. The labels were produced by large language model readers applying a written codebook, not by a panel of human domain experts. This is a real limitation and we state it plainly rather than describing the process as human annotation. Three things mitigate it and one does not. Mitigating: the codebook was written before any monitor, every window was read by a second reader under reduced context with a different reader, and we report that agreement (88.1% binary, $\kappa = 0.70$), including the fact that disagreement concentrates on window boundaries. The process is also fully auditable: every label carries a justification citing step numbers, and the cards are in the released artifact. What it does not provide is an independent human ground truth; a student team should spot-check a stratified subsample against the cards and report any systematic disagreement, and any downstream use of these labels should treat them as a strong consensus annotation rather than as expert adjudication.

Label granularity. Stagnation is graded, but our unit is a fixed window. Windows that cut a productive interval in half are inherently ambiguous, which is the main source of cross-round disagreement and a systematic source of error for every monitor evaluated on a single threshold.

The semantic channel is a surrogate. The intended rolling-diversity baseline uses a pretrained sentence encoder. The model host was unreachable from the execution environment, so we substituted a hashing bag-of-words representation with the same downstream statistics. This makes the SEM numbers a *lower bound* on what an embedding-based monitor could achieve, and it makes cross-paper numeric comparison invalid; the qualitative conclusion (representation similarity is not evidence of task progress) is unaffected, but a reader should not quote our SEM AUC as the state of the art for that family.

Redaction. A substantial fraction of observations in the public release are redacted (22.4% on average). Redaction removes both entities and verification state, so it degrades EVID and VER — the channels we highlight — more than it degrades repetition. The reported results are therefore conservative for the proposed signals, and the direction of the bias is stated here so that it is not mistaken for a strength.

Task grounding is lexical. Relevance is computed from term overlap between an entity and the task statement. It fails when the task is described abstractly and the relevant code uses different vocabulary, and it can be fooled by an entity that shares a common word with the statement. A learned or model-based relevance estimate would likely improve the channel; we did not use one because we wanted the monitor to remain reference-free, cheap, and explainable, and because a model in the monitoring loop would reintroduce exactly the cost the monitor is meant to save.

Statistics. 81 trajectories carry region annotations and 45 of them contain at least one stagnant region, so alarm-level estimates have wide intervals; we therefore report trajectory-level bootstrap intervals and prefer the window-level alarm metric, which uses every annotated window. The operating thresholds are selected on the annotated set, which makes the reported points optimistic; the full curves are reported for that reason. Finally, the per-scaffold breakdown rests on single-digit numbers of trajectories for several scaffolds and should be read as descriptive.

9 Reproducibility

The pipeline is deterministic and split into stages, each of which writes a frozen artifact:

1. `scripts/build_dataset.py` draws the stratified sample and writes window features;
2. `scripts/embed_semantics.py` writes the step representations used by the semantic channel;
3. `scripts/make_annotation_cards.py`, `make_dense_cards.py`, `make_extra_dense_cards.py` write the annotation cards from the sampled trajectories;
4. `scripts/build_gold.py` merges all annotation rounds into `adjudicated.csv`, `agreement.json` and `regions.json`;
5. `scripts/run_experiments.py` runs task-level cross-validation and writes `window_features.parquet`, `monitor_scores.parquet`, the metric tables and the run configuration;
6. `scripts/make_figures.py` and `export_results_tex.py` regenerate every figure and every number in this paper from those artifacts.

Data are public: Terminal-Bench 2.0 trajectories [6] and SWE-agent trajectories [7]. Reviews of one corpus are released under their licenses. The monitor itself is about six hundred lines of dependency-free Python (standard library plus NumPy) and runs in milliseconds per step on one CPU core: measured on 2016 windows it takes 0.33 ms to compute a window’s feature vector and 0.003 ms to score it and evaluate the alarm rule, so monitoring a thousand-step run costs under a second. The only external service used anywhere in the pipeline is the annotation step.

Two automated checks guard the numbers. A test suite (`scripts/run_tests.py`) asserts the invariants the protocol depends on — in particular that truncating a trajectory leaves the features of the surviving windows unchanged, which is the online restriction, and that the annotation file is internally consistent. A number audit (`scripts/audit_numbers.py`) re-derives the headline quantities straight from the frozen artifacts and compares them with the macros the paper prints; at the time of writing all 42 checked values matched.

10 Conclusion

We asked which observable signals let an online monitor recognise that a coding agent has stopped making progress, without knowing the correct solution. The answer our data supports is narrower

and more useful than "detect loops". Exact repetition is a precise but partial detector; semantic redundancy of successive steps is a good ranking signal but blind to direction; workspace churn is close to worthless; verification deltas are rare but nearly decisive; and the signal that adds the most is the cheapest to compute — whether the material the agent is currently uncovering is new *and* about the task. That last finding is the one we would want a runtime builder to take away, together with the negative results: the signals that look most natural are not the ones that work, and a monitor that stops healthy runs is worse than no monitor at all.

11 Author contributions and use of AI tools

Both authors contributed to the conception of the study, the design of the codebook, the annotation protocol, the interpretation of the results and the writing of this report. The implementation, the annotation pipeline and the first drafts were produced with an autonomous AI coding agent; its role, the stages it touched, the tools and versions involved and the errors it made are recorded in full in the accompanying AI-assistance log, which is submitted as supporting material together with the annotation cards, the frozen feature matrices and the monitor implementation. No result reported here was taken on trust from that log: every number in the paper is regenerated from the frozen artifacts by the released scripts (Section 9), and the label set remains open to independent human re-reading.

A Budget-calibrated operating points

The main text reports the raw threshold sweep, in which every monitor uses the same score threshold. That view favours monitors whose scores happen to sit on a convenient scale, so we also calibrated each monitor’s threshold on the training trajectories: for a firing budget b , the threshold is the $(1 - b)$ quantile of that monitor’s training step scores, so every monitor fires at a comparable rate before the persistence rule is applied. Under this calibration the ordering changes and detection collapses for most monitors: at the 5% budget the hand-designed evidence–semantic monitor catches 0.101 of annotated stagnant windows with 0.007 false stops, and the best learned semantic mixture reaches 0.130 with 0.002. The reason is informative rather than disappointing: the extreme scores inside a run are frequently *productive* moments — the one decisive discovery, the test that finally passes — so a monitor tuned to fire only at its own extremes is tuned to fire in the wrong place. We therefore report both views and treat the raw sweep as an optimistic bound.

B Annotation codebook

The codebook given to annotators is reproduced verbatim in the released artifact as `docs/annotation_guide.md`. Its core is the three-channel progress test (§3): a window is productive if new task-relevant information became known, or a plausible artefact changed durably, or verification state improved; it is stagnant if none of those happened while the agent kept acting. Six labels are available, of which two (`BLOCKED__EXTERNAL`, `UNCERTAIN`) are held out of the binary task.

C Window-size sensitivity

The window length w is the one hyperparameter the feature set has, so we re-ran the whole protocol at $w = 6, 20$ and 30 . The window matters, and it is not monotone. At $w = 6$ the monitors have too

little context: the semantic family falls to 0.565, and per-window detection at a 5% budget drops to about a fifth of its $w = 10$ value. At $w = 20$ the global ranking is similar (the semantic family reaches 0.759) but the within-run numbers move around a lot, and at $w = 30$ the window is long enough that productive work inside it masks earlier stagnation, which costs the evidence channel heavily. The $w = 10$ operating point used throughout is thus a compromise between context and resolution rather than an optimum we would defend on principle; a deployment should tune it, and the artifacts contain the full comparison.

D Cross-scaffold generalisation

Does a monitor fitted on some agent scaffolds work on one it has never seen? We trained the same feature sets on annotated windows from every scaffold except one and evaluated on the held-out scaffold, for each of the nine scaffolds in the sample. The ordering from the main experiments survives, but the numbers are lower and the spread is wide: the semantic family averages 0.679 (range 0.519–0.833), the arithmetic train-and-evaluate result for the same family inside the corpus is 0.742, and every family has at least one scaffold where it is close to chance. Repetition (0.594) and workspace (0.615) transfer about as well as each other, and task-grounded evidence on its own transfers worst (0.557), again improving only in combination with the semantic channel (0.677). The practical reading is that these signals are portable in kind but not in calibration: a deployment should re-fit its threshold per scaffold rather than carry one across agents.

A genuine cross-corpus check — the same extractor applied to SWE-agent trajectories over SWE-bench — was prepared but could not be evaluated here, because that corpus carries no stagnation annotations; reporting alarm rates there without labels would be a claim we cannot support. Adding that annotation set is the first item of future work.

E Channel behaviour, per trajectory

The figure below shows the three channels step by step for three annotated trajectories. Each was chosen because it contains both labels, so the shaded bands show where the same run stopped progressing; the pattern is the one the aggregate statistics report.

References

- [1] Anthropic. Async agent stall watchdog kills healthy long-running requests at 600s timeout. Issue #85265, anthropics/claude-code, 2026.
- [2] Anthropic. Cascading stall-watchdog failures when running 5+ parallel background subagents. Issue #86499, anthropics/claude-code, 2026.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [4] Sanjay Khanna. Zombie agents: Detecting semantic livelock in long-horizon autonomous software. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware)*, 2026. Main track; OpenReview B1VgsZzg0N.

- [5] James Kwok et al. LLM-as-a-Verifier: A general-purpose verification framework. arXiv preprint arXiv:2607.05391, 2026.
- [6] Yoonho Lee. Terminal-Bench 2.0 trajectories. Hugging Face dataset `yoonholee/terminalbench-trajectories`, 2026. Full agent trajectories from the Terminal-Bench 2.0 leaderboard; accessed 2026-09-10.
- [7] Nebius. SWE-agent trajectories. Hugging Face dataset `nebius/SWE-agent-trajectories`, 2024. 80,036 SWE-agent trajectories over SWE-bench and SWE-bench-extra; accessed 2026-09-10.
- [8] Opencode. Agent has no effective no-progress/loop detection during repeated file investigation. Issue #43603, anomalyco/opencode, 2026.
- [9] Opencode. Subagent: infinite loop of identical tool calls for ~50 min with no loop protection. Issue #45442, anomalyco/opencode, 2026.
- [10] Dzung Pham, Kleomenis Katevas, et al. AgentStop: Terminating local AI agents early to save energy in consumer devices. arXiv preprint arXiv:2605.15206, 2026.
- [11] Rui Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv preprint arXiv:2607.06184, 2026.
- [12] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. Fail-fast, restart-smart: Early failure prediction and restart for SWE agentic tasks. arXiv preprint arXiv:2608.03222, 2026.
- [13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.
- [14] Huan Zhang, Mingju Chen, Dongxu Zhou, Can Lv, Heng Chang, Sen Cui, Faguo Wu, and Shiji Zhou. TRCA: Transition-wise rubric credit assignment for long-horizon LLM agents. arXiv preprint arXiv:2608.16156, 2026.
- [15] Xu Zhao et al. Failure as a process: An anatomy of CLI coding agent trajectories. arXiv preprint arXiv:2607.09510, 2026.

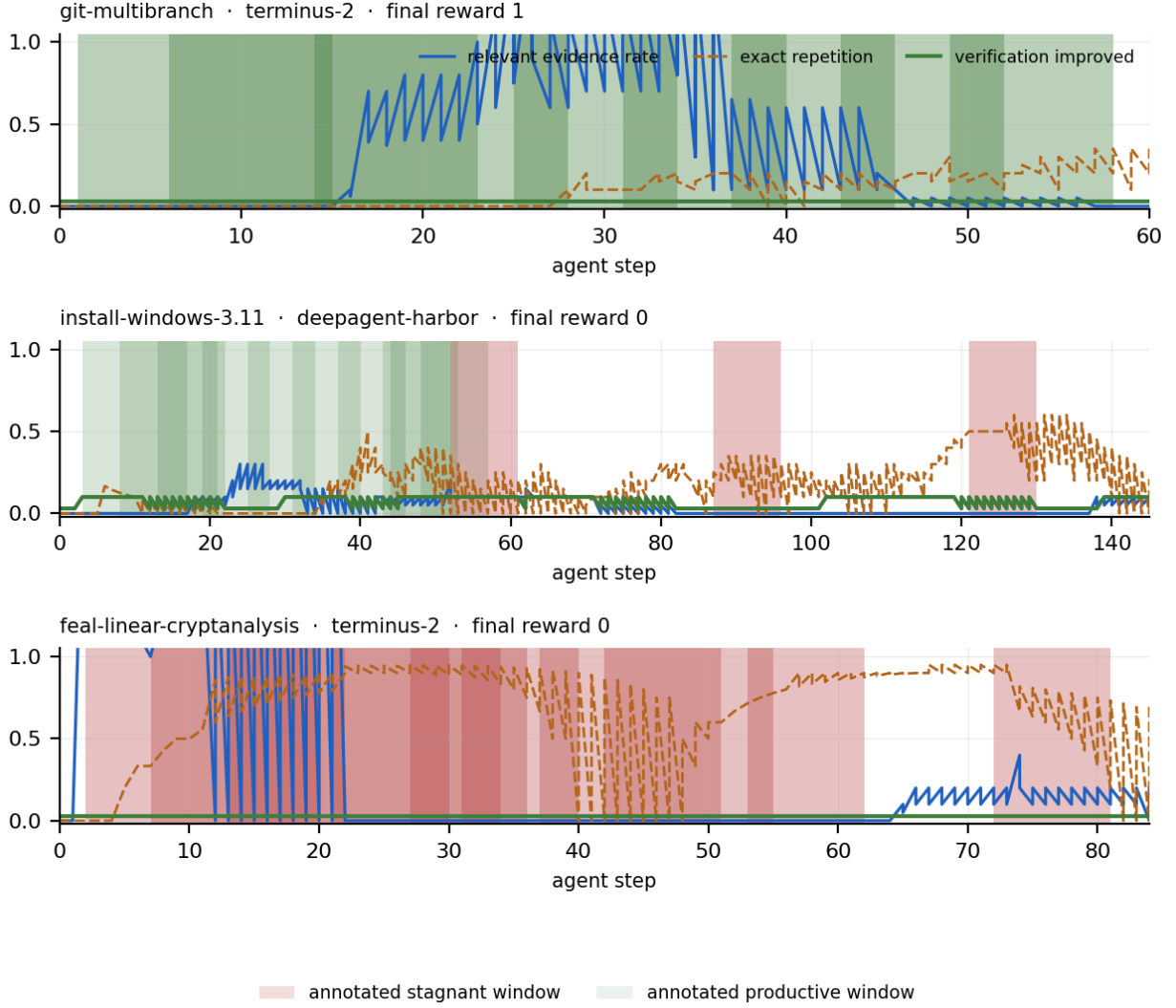


Figure 7: Channel behaviour on three annotated trajectories, chosen for contrast: each contains both labels, so the shaded bands show where the same run stopped progressing. The relevant-evidence rate falls inside the red bands while exact repetition stays flat — the pattern that makes repetition-based guards miss stagnation — and the green ticks mark the windows in which verification state moved.